Task Manager App (React + Vite)

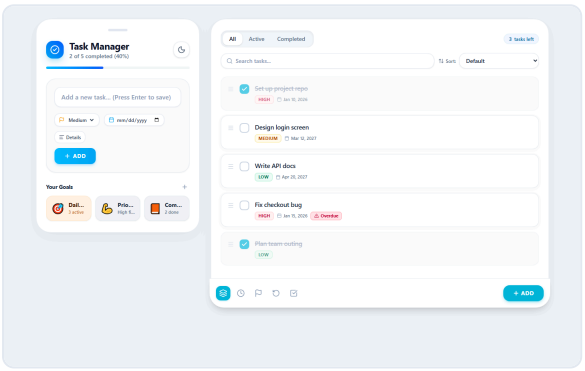
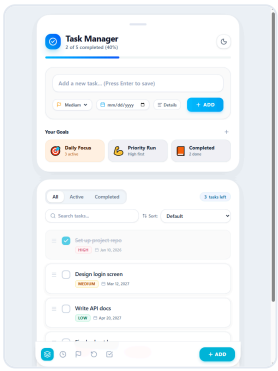
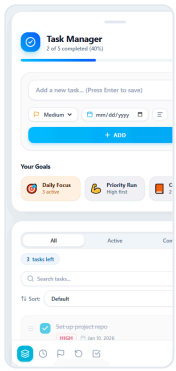
A responsive, accessible, single-page Task Manager application built with **React (v19)**, **Vite**, and **Tailwind CSS**. Built as part of the Frontend React Intern Assessment for **Hyperapps**.

🚀 Live Demo & Repository

- **GitHub Repository:** https://github.com/Ayush-97techyboy/hyperapps-task-manager
- **Live Preview:** Available via Vercel / Netlify / `http://localhost:5173/`
- **Stack:** React 19, Vite, Tailwind CSS v4, Lucide React, Vitest

---

📸 Responsive Screenshots

Desktop View (960px – 2560px)	Tablet View (768px)	Mobile View (375px)
		

🛠️ How to Run the Project

The application requires Node.js (v18+) and runs with standard npm scripts with **zero extra setup**:

```
# 1. Install dependencies
npm install

# 2. Start development server
npm run dev
```

The app will be accessible at `http://localhost:5173/`.

Run Unit Tests

```
# Run Vitest unit test suite (validates filter, sort, and starter data checks)
npm test
```

Production Build

```
npm run build
npm run preview
```

✨ Features Implemented (All 9 Must-Haves + Bonuses)

1. First Load & Fetch with Retry

- Loads starter tasks from `public/tasks.json` via asynchronous `fetch()`.
- Displays an accessible animated loading spinner (`LoadingState.jsx`).
- Handles fetch errors gracefully with an intuitive error banner and a **Retry** button (`ErrorState.jsx`).

2. LocalStorage Persistence

- Every change (adding, toggling, editing, deleting, or undoing tasks) persists automatically to `localStorage` under the key `hyperapps_tasks`.
- Subsequent visits instantly restore state from `localStorage` without re-fetching stale JSON.

3. Add Task

- Clean task creation form with:
  - **Title** (required; validates against empty strings or whitespace-only inputs with visible red alert).
  - **Priority selector** (`Low`, `Medium`, `High`; defaults to `Medium`).
  - **Optional Due Date** picker (`YYYY-MM-DD`).
- **Enter key submits** seamlessly.

4. Task List & Rendering

- Displays task title, priority badge, due date, and accessible checkbox.
- Completed tasks feature strikethrough styling and muted opacity.
- Color-coded priority badges:
  - 🔴 **High:** Rose
  - 🟡 **Medium:** Amber
  - 🟢 **Low:** Emerald

5. In-Place Inline Editing

- Click on any task title or the edit pencil icon to enter edit mode.
- **Enter** saves the edited title.
- **Escape** cancels and reverts to the original title.
- Blank or whitespace-only titles are strictly prevented with inline validation feedback.

6. Delete with Confirmation Modal

- Clicking the trash button triggers a custom accessible dialog (`ConfirmDialog.jsx`) with Cancel and Delete options.
- Supports keyboard navigation (`Escape` to dismiss).

7. Filter Tabs & Dynamic Counter

- Tabs for **All**, **Active**, and **Completed**.
- Live dynamic counter computing active tasks left (e.g., `3 tasks left` or `1 task left`).

8. Search & Multi-Criteria Sorting

- **Case-insensitive live search** by title that works seamlessly alongside the active status filter.
- **Sorting options**:
  - **Due Date:** Soonest first; tasks with no due date (`null`) appear last.
  - **Priority:** High $\rightarrow$ Medium $\rightarrow$ Low.
  - **Default:** Preserves order of creation.

9. Overdue Indicator & Mobile Responsiveness

- Incomplete tasks whose due dates are in the past display a pulsing red **"Overdue"** badge.
- Fully responsive layout engineered for **375px-wide mobile screens** up to 4K desktop displays.

🎁 Bonus Features Included

1. **5-Second Undo Toast:** When a task is deleted, a non-intrusive bottom-right toast appears with an **Undo** button. Restores the task to its exact position if clicked within 5 seconds.
2. **Dark / Light Theme Toggle:** Accessible theme toggle saved to `localStorage` (`hyperapps_theme`) and synced with system preferences, configured with Tailwind CSS v4 `@custom-variant dark` for full `.dark` class selector support.
3. **Automated Tests (Vitest + React Testing Library):** Comprehensive test suite across 4 test files (`taskUtils.test.js`, `TaskControls.test.jsx`, `TaskInput.test.jsx`, `useTheme.test.js`) validating starter data constraints, filter/sort logic, component interaction, and theme switching.
4. **Accessibility (a11y):** Semantic HTML5 (`<header>`, `<main>`, `<section>`, `<aside>`), real `<button>` elements, ARIA labels, live regions, and visible keyboard focus rings.

🧪 Starter Data Check Verification

The unit test suite in `src/utils/taskUtils.test.js` validates all criteria with the starter data:

1. **Counter:** Shows **3 tasks left** (Tasks with id 2, 3, 4 are active).
2. **Overdue:** Only `"Fix checkout bug"` (id 4, due `2026-01-15`, incomplete) shows **Overdue**. Task 1 (`2026-01-10`) is completed, so it is **not** overdue.
3. **Due Date Sorting on 'All' Tab:**
    1. *Set up project repo* (`2026-01-10`)
    2. *Fix checkout bug* (`2026-01-15`)
    3. *Design login screen* (`2027-03-12`)
    4. *Write API docs* (`2027-04-20`)
    5. *Plan team outing* (`null`, placed last)

🏛️ Architecture & Component Hierarchy

Adheres strictly to the unidirectional data flow guidelines:

```
src/
├── components/
│   ├── ConfirmDialog.jsx   # Accessible modal for delete confirmation
│   ├── ErrorState.jsx      # Error banner with retry mechanism
│   ├── Header.jsx          # App branding, completion stats & theme toggle
│   ├── LoadingState.jsx    # Fetch loading spinner
│   ├── TaskControls.jsx    # Status tabs, counter, search bar & sort select
│   ├── TaskInput.jsx       # Task creation form with validations
│   ├── TaskItem.jsx        # Individual task row with inline edit & badges
│   ├── TaskList.jsx        # Container with empty-state illustrations
│   └── UndoToast.jsx       # 5-second undo toast notification
├── hooks/
│   ├── useTasks.js         # Single source of truth for tasks, fetch & localStorage
│   └── useTheme.js         # Theme switcher with localStorage synchronization
├── utils/
│   ├── taskUtils.js        # Pure filter, sort, overdue & formatting functions
│   └── taskUtils.test.js   # Vitest unit test suite
├── App.jsx                 # Top-level orchestrator & state composition
├── index.css               # Tailwind CSS entrypoint
└── main.jsx                # React DOM root mounting
```

- **One Source of Truth:** `tasks` array managed in `useTasks.js`.
- **Zero Derived State Mutation:** Filtered list, sorted results, and remaining counters are computed dynamically using `useMemo` without mutating the original state.

🔍 What is Missing or Broken

- **Core Requirements: Nothing is missing or broken.** All 9 must-have requirements, starter data checks, and all bonus requirements are fully implemented, functional, and backed by automated tests.
- **Edge Cases Handled:**
  - Empty / whitespace-only task titles show immediate validation feedback.
  - Deletions are safely guarded by a confirmation dialog with a 5-second undo toast.
  - Sorting and filtering update reactively without mutating original state.
  - Responsive design tested down to 375px mobile viewports.
- **Out of Scope (By Design):**
  - No backend or database (per assessment rules: React state + `localStorage` only).

⏱️ Time Spent & Assumptions

- **Time Spent:** ~2.5 hours total (scaffolding, component separation, unit testing, styling, and documentation).
- **Assumptions:**
  - Standard YYYY-MM-DD ISO date format for task due dates.
  - Overdue status only triggers on incomplete tasks whose due date is strictly before the current day.
  - When sorting by due date, tasks without a due date are gracefully ranked at the bottom.